

2026 年 5 月 22 日 Version x.x

ハッカソン ～計画書と設計書～

チーム 2

24G1146 : 川又 快斗

24G1053 : 熊本 朱純

24G1133 : 安尾 晃貴

25G1084 : 陳 夏薇

25G1089 : 常世田 隆一

25G1110 : 早川 和輝

25G1115 : 福島 健一

目次

第 1 章	プロジェクトの計画書	1
1.1	検証と妥当性確認: V&V 計画	1
1.1.1	同期制御での有効指標 MOE	1
1.1.2	同期制御のシステムテストで評価する性能指標 MOP	1
1.1.3	単体テストで評価する性能指標 MOP	1
第 2 章	システムの詳細設計	3
2.1	部品一覧	3
2.1.1	同期制御の部品	3
2.2	ソフトウェア設計	3
2.2.1	同期制御のソフトウェア設計	3
2.2.2	関数設計	4
2.3	処理フローの設計	4
2.3.1	同期制御の処理	4
2.4	テストの設計	5
2.4.1	同期制御のテスト	5

第 1 章 プロジェクトの計画書

1.1 検証と妥当性確認: V&V 計画

1.1.1 同期制御での有効指標 MOE

同期制御における有効指標の 1 つ目として，システムの実用化のための通信の高い信頼性である．システムの通信の信頼性が高くなければ，通信が失敗してしまい実用的とは言えない．よって，Arduino 間での通信の信頼性を高めて実用的であることを評価する．

2 つ目はとして，通信の遅延についてである．本プロジェクトでは輪唱曲を演奏するため，楽器の一つ一つをタイミングよく演奏する必要がある．よって，通信によって発生する遅延を小さくして違和感なく輪唱曲を演奏することを評価する．

1.1.2 同期制御のシステムテストで評価する性能指標 MOP

通信の信頼性を確かめるために，通信成功率のテストを行う．通信が行われた際に，シリアルモニタに通信成功を表示するようにプログラムを作成してテストする．また，遅延の秒数を調べるために通信が完了した際に信号を返し，通信の遅延を調べる．

1.1.3 単体テストで評価する性能指標 MOP

通信において，通信の成功率が 98%以上であれば信頼性は高いという指標があるため [1]，通信成功率 98%を目標として設定する．また，遅延については 20ms 以下であれば人間は遅延を感じないという指標があるため [2]，通信の遅延は 20ms 以下を目標とする．

第 2 章 システムの詳細設計

2.1 部品一覧

2.1.1 同期制御の部品

同期制御では、通信の親機としての Arduino を 1 台で実装を行うが、楽器演奏の Arduino を子機として受信するため楽器演奏の Arduino では受信するためのプログラムを実装する。また、本プロジェクトでは無線通信を用いるため電源供給以外にはタイプ C ケーブルは使わない。

2.2 ソフトウェア設計

2.2.1 同期制御のソフトウェア設計

通信方式として、ブロードキャスト、ユニキャスト、マルチキャストがあるが本プロジェクトではマルチキャストを用いる。ブロードキャストは、Wi-Fi 内のすべての機器に通信してしまうため関係のない危機にまで通信を行ってしまう。次に、ユニキャストは 1 対 1 の通信であるため、複数の Arduino に通信を行うためには複数回通信を行わなければならない。その際、通信に遅延が生まれてしまう可能性があるためユニキャストは用いない。そして、マルチキャストは Wi-Fi 内のグループを作成した一部の機器と通信を行うことができる。よって、本プロジェクトではマルチキャストを用いる。

また、通信の信頼性を上げるために重複送信を行う。通信する際に、同じ内容の packets を数ミリ秒の感覚を開けて送信することで信頼性を高めることができると考え、重複送信を実装する。重複した信号を受け取らないために、受信側では一度受信したら一定時間同じ packets を受信しないプログラムの実装も必要となる。

2.2.2 関数設計

マルチキャストパケット一斉送信関数の `sendSyncPacket()` で入力信号やリズム値をパケットに詰め、マルチキャストアドレスに対して UDP で一斉送信する。信頼性向上のため、同じパケットをパケット ID を維持したまま短時間で 2 回連続送信する。引数としては入力信号である "0" と "1" , そしてリズム値である。戻り値としては、送信処理が成功したかどうかの成否のフラグである。

受信側の子機では、パケット受信・重複チェック関数として `receiveAndFilterPacket()` を実装し、UDP マルチキャストパケットを受信する。また、パケット内を読み取り、前回処理したのと同じ重複パケットであれば破棄するものである。引数はなく、戻り値は新規パケットであれば "true", 重複であれば "false" とする。

そして、輪楽曲であるためタイミングをずらすための関数として `playNextNote()` を実装する。これは、親機から届いたリズム値を基に、自分の楽器が鳴らすべきタイミングに達していれば演奏を始めさせるものである。引数としては、親機からのリズム値であり、戻り値はない。

2.3 処理フローの設計

2.3.1 同期制御の処理

同期制御の処理の手順を図 2.1 に示す。このようにセンサ制御から送信された際に楽器演奏にタイミングをずらしながら順番に通信して重複送信を行う。また、センサ制御からリズム変更の値が送信された際には楽器演奏にリズム変更の値を送信する。

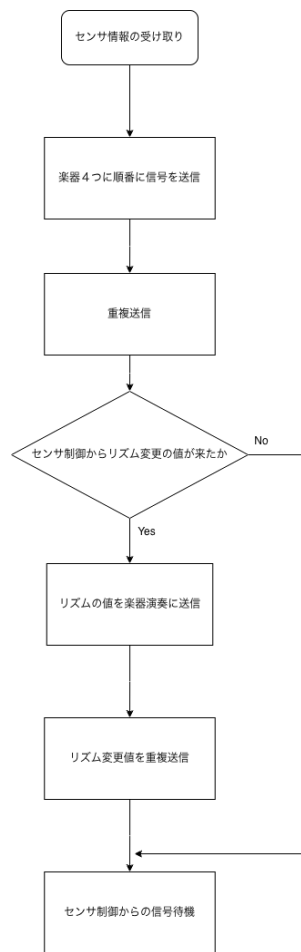


図 2.1 同期制御の処理のフローチャート

2.4 テストの設計

2.4.1 同期制御のテスト

評価指標として示した，通信成功率 98%以上であることを確かめるために，通信が行われた際にシリアルモニタに通信の成功を表示するテストを 100 回行い 98 回以上成功であることを確かめる．また，通信の遅延のテストは，通信した際にテストとしてシリアルモニタに遅延の秒数を表示させて 20ms 以下であることを確かめる．

参考文献

- [1] Khujamatov, H.; Davronbekov, D.; Khayrullaev, A.; Abdullaev, M.; Mukhiddinov, M.; Cho, J. "ERIRMS Evaluation of the Reliability of IoT-Aided Remote Monitoring Systems of Low-Voltage Overhead Transmission Lines". *Sensors* 2024, 24, 5970.
- [2] 長尾翼, 渡邊珠希, 池田雄介, 上野佳奈子, 伊勢史郎. "音の遅延条件がアンサンブル演奏に与える影響に関する検討." *音日本音響学会講演論文集 (春)* (2012): 2-6.